



TRUSS

A coordination layer for shared electrical capacity

Five members. Thirty devices. One hard limit — and it will not lie about it.

6

safety invariants

5

falling members

48h

execution plan

A truss carries a load no single beam could,
by giving every member only what it can bear.

CONTENTS

What is in here

What is in here	2
The decision	3
The name	5
The problem	7
Where this sits	9
The mechanism	11
Safety	15
Architecture and protocol	18
Scope and proof	21
The console	24
Execution	26
The demo	29
Pitch and defence	31
Risks	33
Sources	35

The three sentences that matter

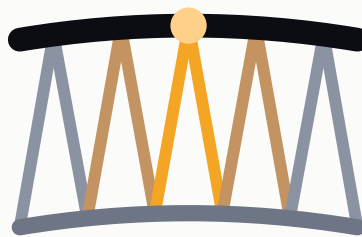
1. The flexibility is already there. What is missing is a way to express it without surrendering privacy or control.
 2. A budget is not a command — it is a lease, and an unrenewed lease expires into safety.
 3. A system that cannot meet its constraint must say so, not fake it.
- Everything else in this document is consequence.

SECTION 01

The decision

Truss is a coordination layer for shared electrical capacity, built as a distributed system of independently failing agents under one hard constraint.

A group of homes, hostel rooms or small buildings shares a constrained feeder, inverter or microgrid. Each **member** knows which of its loads are critical, which can wait, and which have deadlines. Members publish only a **flexibility offer** — never a device list. A **coordinator** divides the available power by a provably fair rule and issues **leases**: an amount, valid until an instant. Members command their own devices, acknowledge, and are measured. Unrenewed leases expire into a safe floor without anyone being told.



The mark is the architecture. Top chord: the cap. Bottom chord: the floor every member keeps. The web between them: the allocation, five panels for five members. The web touches the top chord and stops — nothing crosses it.

Why this is a large project and not a dashboard

Five genuinely hard things at once, and every one of them is visible in the demo.

	What it is	Why it is not decorative
A distributed system	Five processes that fail independently, asynchronous messages that duplicate and reorder, stale state, recovery.	Not one program pretending to be five. Kill any one and watch the rest respond.
A leased resource	The sum of outstanding lease amounts never exceeds the cap — and that holds even if every message in flight is lost.	Gray & Cheriton's lease mechanism applied to watts instead of cache lines. [S01]
A fair allocation	Water-filling produces the max-min fair (leximin) allocation, deterministically, in $O(n \log n)$.	Fairness with a theorem behind it, not an adjective on a slide. [S02]
A cyber-physical loop	Sense, offer, allocate, lease, command, acknowledge, measure, replan.	It cannot be reduced to CRUD screens without ceasing to work.
A standards position	OpenADR 3.x above, MQTT 5 in the middle, Matter 1.5 below.	We implement the middle honestly and define the edges as interfaces we do not claim.

What makes it memorable

Most teams demonstrate that their system works. Truss demonstrates what happens when it does not — and that is a far harder thing to fake.

Kill the coordinator and the site gets quieter.

Every lease counts down on screen. Every member steps to its floor. There is no chaos, because there was never any trust to lose. Restart it and it refuses to issue a lease for one full lease term, because it cannot know what it promised before it died.

Ask for the impossible and it refuses.

Set the cap below the sum of the declared safety floors and the system declares INFEASIBLE and names the members it cannot serve. It never reports a compliance it did not achieve. A system that lies about its constraint is worse than no system, because someone would trust it.

Those two beats are the pitch. They are also the engineering. Neither is available to a team that built a monitoring dashboard, at any level of polish.

Ruthless boundary

Truss is not a utility-grade demand-response platform, a certified smart panel, a microgrid controller, or a savings product. **It never switches mains voltage.** The prototype proves exactly six things, and every one of them is measured:

- 1 Interoperable device messaging under one versioned contract.
- 2 A maintained device twin in which **unknown is not zero**.
- 3 Constraint-aware, provably fair allocation.
- 4 Closed-loop command verification against observed watts.
- 5 Fail-safe degradation under coordinator loss, partition and restart.
- 6 An append-only evidence trail that replays deterministically.

That is already a serious 48-hour system. Everything outside those six is recorded as a rejection with a reason, because rejections are cheaper to defend than features.

SECTION 02

The name

A good system name is a compression of the design. It should keep paying out every time you reach for a word. Truss does that on three levels, and all three are load-bearing.

The etymology is the architecture

Truss entered English from Old French **trousse** — a bundle: separate things bound together and carried as one. That is the distributed-systems claim sitting inside the word. Five independent members, bound by a protocol, behaving as one structure. Not one machine pretending to be five, and not five machines pretending to be one.

The structure is the mechanism

A truss carries a load that no single member could carry, by **resolving** it into forces along members that each stay inside their own limit. Two properties follow, and both are properties we want.

- **Every share is computable, not guessed.** The method of joints gives the exact force in every member. Truss gives the exact watt budget for every home, by a stated rule, with the arithmetic on screen.
- **The load path is traceable.** A structural engineer follows a load from where it is applied to where it is carried. Truss does the same in reverse: click any deferred device and get the causal chain that deferred it.

There is a third property, and it justifies the privacy boundary mathematically. The **method of sections** lets you cut through a truss and compute the forces crossing that cut without solving the rest of the structure. Truss cuts at the member boundary: the coordinator reasons about five offers, not thirty devices. Its problem is $O(\text{members})$, not $O(\text{devices})$ — which is the privacy argument and the scaling argument, and they turn out to be the same argument.

Redundancy is the engineering word for fault tolerance

A planar truss with j joints is *statically determinate* at $m = 2j - 3$ members. Below that it is a mechanism and it collapses. Above that it is **statically indeterminate** — it has spare load paths, so losing a member redistributes force rather than ending the structure.

Structural engineers have called this *redundancy* for two centuries. It is the same property distributed-systems engineers call fault tolerance — discovered independently, in the same century, for the same reason.

Truss is deliberately indeterminate. Kill a gateway and the remaining members absorb its share.

The vocabulary

The name handed us these, and every one replaced a worse word. This is not branding varnish: a team that shares precise vocabulary argues faster and builds fewer misunderstandings.

Truss term	System meaning	What it replaced
Member	One home, room or gateway. Structural <i>and</i> social — both readings are correct.	“node”, “client” — neither suggests belonging
Top chord	Site capacity ceiling. Hard. Never crossed.	“cap”, “limit”
Bottom chord	The floor every member keeps.	“minimum”, “critical load”
Web	The live allocation between the chords — the part that moves.	“the schedule”
Joint	The coordinator; where members meet and forces resolve.	“server”, “master”
Camber	Headroom reserved for load we cannot currently see.	“safety margin”, “fudge factor”
Load path	The causal chain behind one decision.	“explanation”, “reason code”
Buckling	Failure by a mode you did not design for.	“edge case”
Gusset	The joint plate — where real trusses actually fail.	“the protocol”, “the contract”

Two of these deserve a paragraph

Camber — a bridge truss is fabricated with a deliberate upward curve so that under full design load it settles *level*. The camber is not error; it is designed-in compensation for a load you know is coming but cannot see yet. Truss reserves a camber band under the top chord for exactly that. When a member goes dark, its last known draw stays reserved, the band grows on screen, and the allocatable power shrinks. An invisible safety principle becomes a visible rectangle.

Gusset — the I-35W bridge in Minneapolis collapsed in 2007 because of undersized gusset plates. The members were fine; the *joints* failed. The lesson transfers exactly. Our members are easy. The protocol between them is where this project will fail if it fails, so that is where the versioning, the idempotence and the property tests go.

Naming rules

- The system is **Truss**. One word. Never “Truss OS”, never “TrussGrid”, and never an acronym expansion — there is not one, and inventing one would be a lie.
- Written **Truss** in prose; **TRUSS** only in the wordmark and slide titles.
- The MQTT namespace is `truss/v1/...`, the CLI is `truss`, the repository is `truss`.
- A home is a **member**, never a “node”. The cap is the **top chord** once the diagram is drawn, and plain “cap” before it.

SECTION 03

The problem

One operating moment

A hostel block loses grid supply at 19:40. The inverter takes over. Normal draw across the block is about 4.6 kW. The inverter can hold 1.8 kW.

Exactly three things can happen today.

- 1 **The inverter trips on overload.** Everyone loses everything — including the corridor lights, the water pump and the router.
- 2 **The warden cuts a floor.** Forty rooms lose everything so that forty other rooms lose nothing. The choice of floor is not a policy; it is whoever the warden thought of.
- 3 **Somebody walks the corridors unplugging things.** This is the humane option and it takes twenty minutes, which is nineteen minutes too long.

All three are blunt instruments applied to a problem that has structure.

The structure nobody uses

Most of that 4.6 kW is not urgent, and the building already knows it. The knowledge is simply stranded at the edge.

Load	The real constraint
Water heater	Needs 40 minutes of heat before 06:00. Does not care when.
Washing machine	Has a deadline , not a schedule. Once started, must not be interrupted mid-cycle.
Laptop charger at 82%	Can pause for an hour with no consequence at all.
Refrigerator	Can coast 20 minutes. Cannot coast 3 hours.
Router, corridor lights	Small, constant, and everything else depends on them.
CPAP machine	Cannot wait one second — and nobody will say which room it is in.

The insight

Every one of those distinctions already exists. None of them is expressible to whatever decides who loses power.

The flexibility is already there. What is missing is a way to express it that does not require surrendering privacy or control.

That final clause is why this is not solved by a smarter breaker. A device list is an intimate document — it says who is home, who is ill, who does laundry at two in the morning. No hostel resident and no apartment neighbour should have to publish one to keep their fridge running. Any design that requires it will be refused by the people it is for, and rightly.

Who this is for

- **Hostels and dormitories** on inverter or generator backup, where a warden currently makes these decisions by hand and by memory.
- **Apartment blocks** sharing a service transformer, where simultaneous EV charging and evening cooking collide.
- **Small institutional microgrids** — clinics, schools, field offices — that island during outages and must keep specific loads alive.
- **Distribution utilities**, one layer up, who want a flexibility envelope from a building rather than switching authority over its appliances.

Why now, and why in India

India is installing the measurement layer and has not yet built the coordination layer above it.

- The **RDSS** programme targets **25 crore smart meters** with an implementation deadline of **March 2028**, against roughly ₹2.83 lakh crore of sanctioned investment. That is a nationwide rollout of devices that can *observe* consumption. [S12]
- Observation is not coordination. A meter that reports a 4.6 kW draw against a 1.8 kW inverter has told you that you are failing. It has not decided, acted, or proved recovery.
- The IEA's work identifies the same gap globally: flexible demand is systematically under-used, and grid-interactive buildings are named specifically as an Indian opportunity joining efficiency, digital technology and peak shifting. [S09][S10][S11]

Truss is the layer that turns the meters India is already installing into something that can act — fairly, and provably.

What “fair” has to mean here

Fair cannot be a vibe on a dashboard. In this system it has three concrete commitments, and each one is testable.

- 1 A floor nobody crosses.** Every member declares a safety floor. While the system reports feasibility, no member is allocated below it. If the cap cannot cover the sum of floors, the system says so and names the members it cannot serve — it does not quietly pick losers.
- 2 Equal treatment of equal need.** Surplus above the floors is allocated by water-filling, which maximises the smallest allocation first. Two members with identical offers get identical budgets. Always, and by construction.
- 3 Sacrifice is remembered and repaid.** A member curtailed heavily in one round carries **debt**, and debt weights the next allocation. Nobody is the designated loser twice.

Those three are Elinor Ostrom's design principles for common-pool resources — clear boundaries, congruence between rules and local conditions, monitoring, and graduated response — rendered as an allocation function. [S08] The commons literature is not decoration here. It is the reason the fairness rule has the shape it has, and it is the answer when a judge asks why we did not simply sort by priority.

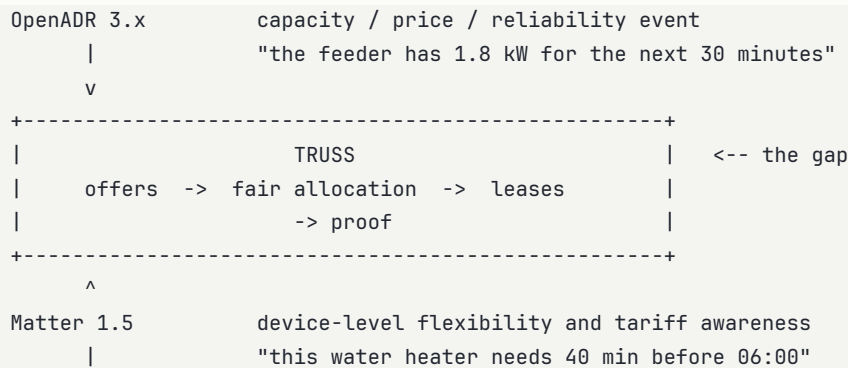
SECTION 04

Where this sits

The fastest way to lose a judge is to describe a solved problem. Here is what exists, stated fairly, and the specific thing each one does not do.

What exists	What it genuinely does	What it does not do
Smart panels SPAN, Lumin	Circuit-level control, load priority, dynamic EV charging, local operation. Real products. [S13][S14]	Optimise one property , behind proprietary hardware , with the panel owner holding switching authority. No notion of independent households negotiating a shared limit.
Home energy platforms Home Assistant	Multi-brand visibility, device integration, local MQTT with discovery. Excellent at this. [S15][S16]	Visibility and user-authored automations. No allocation across parties, no fairness, no lease safety. Rules are written per home rather than negotiated between homes.
Utility demand response OpenADR 3.x	Standardised exchange of price, reliability and capacity events between utilities, aggregators and control systems. REST with webhooks. [S05]	Stops at the building boundary. It signals <i>that</i> capacity is constrained; it does not decide which of your neighbour's loads yields, or prove that it did.
Matter 1.5	New energy device types and clusters — Commodity Tariff, Commodity Price, Electrical Grid Conditions, Energy Preference, Power Topology — so devices report true cost and adjust. [S06]	Device interoperability, not allocation. It makes flexibility <i>expressible per device</i> . Something still has to decide, fairly, across many owners. That is the gap.
Rotational load shedding	Universally deployed, well understood, zero infrastructure.	Coarse to the point of cruelty. It cuts by geography, not by need, and cannot tell a CPAP machine from a water heater.
Packetized Energy Management academic and deployed	Devices make anonymous asynchronous requests for fixed-length energy packets; a coordinator grants or denies. Statistically fair grid access, preserved privacy and quality of service. [S07]	The closest prior art, and we say so out loud. It coordinates a fleet of like devices for a utility. It does not model a household boundary with its own floor, deadlines and accumulated debt, and it provides no per-member safety lease.

The layer nobody owns



The four things that are actually novel

The defensible novelty is not “energy management”, which exists. It is this combination, which does not currently appear together anywhere we could find:

- 1 A household privacy boundary that is also the scaling boundary.** Members publish offers, not device lists. The coordinator's problem is $O(\text{members})$.
- 2 A safety lease rather than a command.** The cap holds under coordinator death, network partition and total message loss.
- 3 Fairness with memory.** Curtailment debt carries between rounds, so the same member is not the designated loser twice.
- 4 Refusal.** When the constraint cannot be met, the system says so and names who it cannot serve, rather than reporting a compliance it did not achieve.

What we must never claim

That we implement OpenADR or Matter. We ship one OpenADR-*shaped* JSON fixture and a Matter adapter *interface*. Both are labelled as such in the UI and the README.

That we are more efficient, cheaper or greener than any named product. We have measured none of that and will not imply it.

That Packetized Energy Management is ours. It is prior art, it is good, we cite it — and being able to name your nearest neighbour in the literature is a strength in front of a technical judge, not a weakness.

SECTION 05

The mechanism

1 — Offers, not device lists

A member aggregates its own devices into one offer. Five numbers describe a household well enough to allocate to it, and nowhere near well enough to know anything about the people in it.

```
{
  "schema": "truss.offer.v1",
  "seq": 1043,
  "member_id": "block-c",
  "floor_w": 180,           // below this the member is harmed
  "firm_w": 420,           // in use, would rather not lose
  "useful_w": 1650,        // most it could productively absorb
  "deadline_energy_wh": 900, // must receive before deadline_by
  "deadline_by": "2026-09-07T06:00:00+05:30",
  "debt_wh": 240,          // curtailment absorbed this session
  "device_count": 6,
  "ts": "2026-09-07T19:41:02+05:30"
}
```

`device_count` exists only so the console can show fleet health. The allocator never reads it. The coordinator cannot leak a device list it was never given — that is an architectural guarantee rather than an operational promise, which is the only kind worth making.

2 — Debt-weighted water-filling

The rule, in one sentence

Give every member its floor, then raise all members together at the same rate until each is satisfied or the power runs out.

That is water-filling, and it produces the **max-min fair** allocation: it maximises the smallest allocation, then the second smallest, and so on — the *leximin* order. [S02] Four properties worth stating to a judge:

- **Deterministic.** Same snapshot, same answer, always.
- $O(n \log n)$. Sort by demand, fill in one pass. Microseconds for five members, and still microseconds for five thousand.
- **Provably fair.** Two members with identical offers receive identical budgets. That is a theorem, not a hope.
- **Explainable in one breath.** “We fill everyone’s glass at the same rate until the jug is empty.” A judge understands it faster than we can draw it.

```
allocate(offers, top_chord_w, camber_w):
    allocatable = top_chord_w - camber_w

    # 1. Floors are not negotiable
    floors = sum(o.floor_w for o in offers)
    if floors > allocatable:
        return INFEASIBLE(unserved = members_beyond(allocatable))
```

```

budget = {o.member: o.floor_w for o in offers}
surplus = allocatable - floors

# 2. Debt weighting: those who gave up more are filled faster.
#   Capped at 2.0 so debt can never starve a debt-free member.
w = {o.member: 1.0 + min(o.debt_wh / DEBT_SCALE, 1.0) for o in offers}

# 3. Water-filling over weighted remaining demand
remaining = {o.member: o.useful_w - o.floor_w for o in offers}
while surplus > EPSILON and any(v > 0 for v in remaining.values()):
    active = [m for m in remaining if remaining[m] > 0]
    rate = surplus / sum(w[m] for m in active)
    step = min(min(remaining[m] / w[m] for m in active), rate)
    for m in active:
        take = step * w[m]
        budget[m] += take; remaining[m] -= take; surplus -= take

return Allocation(budget, rule="debt_weighted_water_filling")

```

DEBT_SCALE is a declared constant shown in the UI, not a tuned magic number. The weight cap of 2.0 is a fairness decision and it is deliberate: unbounded debt weighting is its own injustice, because it lets a member's history starve a neighbour who has never been curtailed.

3 — Leases, not commands

A budget is not a command. A budget is a lease.

A command says *do this*, and it is only as good as the channel that carried it. A lease says *you may draw this much until your own clock says stop*, and it is good even if the channel dies, the coordinator dies, and every message in flight is lost.

```
{
  "schema": "truss.lease.v1",
  "plan_id": "plan-0042",
  "version": 42,
  "member_id": "block-c",
  "budget_w": 312,
  "ttl_ms": 6000,           // a DURATION, never a deadline
  "reason": "cap_drop",
  "basis": {               // the load path travels with the decision
    "top_chord_w": 1800,
    "camber_w": 240,
    "allocatable_w": 1560,
    "rule": "debt_weighted_water_filling",
    "members_at_floor": 0
  },
  "correlation_id": "cap-event-0007"
}
```

Three properties, each earning its place

- **ttl_ms is a duration, not a deadline.** The member starts its own monotonic countdown on receipt. Five laptops on a hotspot will never agree on wall time and do not need to.
- **basis is the load path.** Everything the console needs to explain this number travels with the number, so explanations cannot drift from decisions — they *are* the decision.
- **version is monotonic.** A member applies a lease only if its version exceeds the last applied. Duplicates and reordering are normal and safe.

The lease rule, stated exactly

A member may draw up to `budget_w`. When its countdown reaches zero without a newer lease, it reduces to `floor_w` — locally, immediately, and without requiring any message.

Everything in the next section follows from that one sentence.

Why not a market, an auction, or peer-to-peer consensus

All three were considered and rejected, and the reasons are worth having ready.

- **Auctions** need a currency, and introducing money into a hostel corridor changes the ethics of the product. Ability to pay is not the same as need, and we would be rebuilding the thing we set out to replace.
- **Peer-to-peer consensus** is a research project. A shared feeder genuinely has a single physical constraint; modelling it as a distributed agreement problem adds failure modes to solve a problem that physics already solved.

- **Priority sorting**, the obvious choice, fails the fairness test outright: the same member is the designated loser every round, forever. Debt weighting exists precisely to prevent that.

SECTION 06

Safety

The distinguishing engineering in this project. If a technical judge probes one thing, it will be this — and it should be.

The six invariants

	Invariant
I1 — Cap	Sum of observed member draw top chord, after the response window.
I2 — Floor	No member is allocated below its declared floor while the system reports feasibility.
I3 — Lease safety	At any instant, the sum of outstanding lease amounts top chord – camber.
I4 — Honesty	If I2 cannot hold, the system declares INFEASIBLE and names the unserved members. It never reports a compliance it did not achieve.
I5 — Idempotence	Applying the same lease or command twice has the same effect as applying it once.
I6 — Monotonicity	A member never applies a lease older than the one it holds.

I3 is the load-bearing one

It is a statement about what the coordinator has *promised*, not about what it has *observed*. Because the coordinator never over-issues, and members self-limit on expiry, the cap holds under coordinator death, network partition, total message loss, and coordinator restart.

Safety is a property of the protocol, not of the planner. The planner may be wrong, slow, or entirely absent. The cap still holds.

A coordinator that restarts **must not issue any lease for one full TTL**, because it cannot know what it granted before it died. It publishes status: recovering with a countdown, the console shows it, and it then resumes. This is Gray & Cheriton's recovery rule and it is four lines of code. [S01]

Camber — unknown is not zero

A member that stops reporting has not stopped drawing power. Treating silence as zero is the single most dangerous thing this system could do, and it is exactly what a naive implementation does.

Camber is the reserved band under the top chord for load we cannot see:

```
camber_w = SUM last_known_draw(m) for every stale or unacknowledged member m
          + a fixed measurement reserve
```

- 1 Stale members keep their last known draw reserved. They are not free.
- 2 The camber band is **drawn on the console**. When a member goes dark you watch the band grow and the allocatable power shrink.
- 3 Camber is repaid the instant the member reports again — the band shrinks and power returns to the pool, live, on screen.

Failure modes and responses

Failure	Detection	Response	In the demo?
Device ignores a command	Missing ack within window	Reserve its observed draw; replan; mark unverified	Yes
Device refuses (mid-cycle)	result: rejected	Legitimate. Plan around it, do not insist	Yes
Member process dies	MQTT Will message, immediate	Camber grows by last known draw; replan	Beat 3
Member partitioned	Lease expires locally	Member self-limits to floor with no message	Yes
Coordinator dies	Every lease expires	All members to floor within one TTL	Beat 5
Coordinator restarts	status: recovering	No lease for one full TTL, countdown shown	Yes
Broker restarts	Connection loss	Leases expire; members floor; retained state restores	Yes
Floors exceed cap	Allocator returns INFEASIBLE	Declare it, name unserved members, do not fabricate	Beat 6
CP-SAT times out	Wall-clock box	Water-filling plan issued; console names the rule	Yes
Clock skew across laptops	—	Cannot occur. Expiry is monotonic and local; no decision reads a timestamp	Explain if asked

Fault injection is a build deliverable

Not a testing afterthought. It has an owner, it ships, and the fault console is a first-class part of the product.

```
truss chaos kill-member C
truss chaos kill-coordinator
truss chaos partition C --seconds 20
truss chaos drop-acks C --rate 0.5
truss chaos delay-telemetry --ms 3000
truss chaos cap --watts 400          # below the sum of floors: forces INFEASIBLE
```

Every one of these runs during rehearsal. At least three run in front of the judges.

Property-based testing

The cheapest credibility available to this team, and it costs about two hours.

```
@given(offers=offer_sets(), cap=watts(), camber=watts())
def test_never_exceeds_cap(offers, cap, camber):
    result = allocate(offers, cap, camber)
    if result.feasible:
        assert sum(result.budget.values()) <= cap - camber
        assert all(result.budget[m] >= 0.floor_w for m, o in offers.items())
    else:
```

```
assert sum(o.floor_w for o in offers.values()) > cap - camber
```

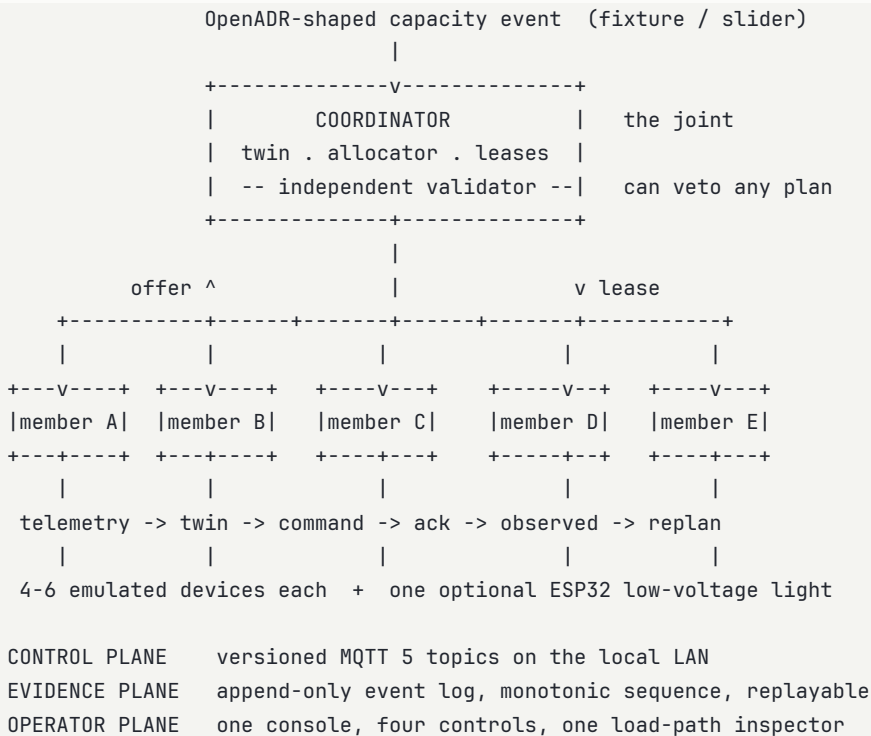
Hypothesis generates thousands of scenarios, including the degenerate ones nobody thinks to write by hand: zero members, one member, every member at floor, identical offers, a cap exactly equal to the floor sum, a cap one watt below it. [S21]

“We fuzz-tested the safety invariant across ten thousand generated scenarios” is a sentence almost no hackathon team can say truthfully. We can, and it takes an afternoon.

SECTION 07

Architecture and protocol

Three planes, kept separate on purpose



The separation is not tidiness. It is what lets the operator plane be rebuilt, the evidence plane be replayed, and the control plane be fuzzed, without any of the three touching the others.

The clock

This is where distributed systems quietly go wrong, so it is decided up front — and it is the first thing a strong technical judge will probe.

- **Lease expiry is measured on the member's own monotonic clock**, never on wall time and never on a timestamp carried in a message.
- The coordinator sends a **duration**, not an absolute deadline. The member starts its own countdown on receipt.
- Message timestamps exist for the event log and for human reading. **No control decision depends on them.**
- Sequence numbers are per-publisher and monotonic. Ordering is decided by sequence number, never by arrival order or timestamp.

Component decisions

Component	Why it exists	Failure fallback
Eclipse Mosquitto	Lightweight local MQTT 5 broker. Sessions, QoS, retained messages and Will messages for abnormal disconnect — the last is what makes member death detectable in milliseconds rather than seconds. [S03][S04]	Broker and core services collapse onto the primary laptop
Member agent	Owens device detail, forms the offer, holds the lease, enforces the floor locally	Last valid budget plus fixed local priority. It keeps working with no coordinator at all
Coordinator	Maintains twins, validates events, calls the allocator, issues and renews leases	Last good plan; on restart, one full TTL of silence
Allocator	Pure function: snapshot to allocation. No I/O, no clock, no network	It <i>is</i> the fallback. CP-SAT sits behind it, never in front
Cap validator	Separate module that independently re-checks any plan before it is issued	If it rejects, the previous plan stands and the UI says a plan was vetoed
Device nodes	Deterministic seeded behaviour; make five laptops behave like five homes	One laptop runs every process with distinct IDs
SQLite event log	Append-only telemetry, offers, allocations, leases, acks, expiries	Newline-delimited JSON file
Console	Streamlit and Plotly — fastest route to a legible operator screen in Python	Static seeded state plus live event counter. Do not rebuild in React mid-event
ESP32 node	Makes one lease physically visible; proves protocol parity	Software node with an on-screen LED on the identical contract

Topic namespace

```

truss/v1/site/{site}/member/{member}/status      retained, Will message
truss/v1/site/{site}/member/{member}/offer
truss/v1/site/{site}/member/{member}/lease
truss/v1/site/{site}/member/{member}/lease/ack
truss/v1/site/{site}/device/{device}/telemetry
truss/v1/site/{site}/device/{device}/command
truss/v1/site/{site}/device/{device}/ack
truss/v1/site/{site}/event/capacity
truss/v1/site/{site}/plan                          retained, latest plan

```

Members subscribe only to their own lease topic. Devices subscribe only to their own command topic. Nothing subscribes to everything except the logger, and the logger is read-only by construction.

Command and acknowledgement

```

{ "schema": "truss.command.v1", "plan_id": "plan-0042", "version": 42,
  "device_id": "washer-1", "desired_state": "deferred",
  "expires_at_ms": 4000, "reason": "member_budget_exceeded",

```

```
"correlation_id": "cap-event-0007" }  
  
{ "schema": "truss.ack.v1", "plan_id": "plan-0042", "version": 42,  
  "device_id": "washer-1", "result": "applied",  
  "observed_state": "deferred", "observed_w": 0, "seq": 88213 }
```

Device rule: apply only if version exceeds the last applied version and the command has not expired; then publish exactly one acknowledgement carrying the *observed* state. Duplicate delivery is normal and must be safe.

result is one of applied | rejected | duplicate | expired. A device that rejects must say why — a washing machine mid-cycle with min_run_s remaining is **allowed** to refuse, and the coordinator must plan around a refusal rather than insist. Members are autonomous; that is the point of the architecture, and it has to be true in the protocol, not merely in the pitch.

Message hygiene

- MQTT 5 **message expiry interval** on commands and leases, so the broker itself drops stale control messages during a partition.
- **Will messages** on member status, so death is detected by the broker rather than by a timeout.
- **Retained** on status and plan, so a joining console is immediately correct.
- QoS 1 everywhere, with idempotence carrying the correctness. QoS 2 buys nothing once versioning is right and costs latency.

Security boundary, stated honestly

The judged build uses explicit demo credentials on an isolated LAN with no public exposure. A production system would need per-device identity, TLS, broker ACLs, key rotation, signed firmware, audit retention and safety-certified actuation. We have built none of those and will say so before a judge has to ask. What we *do* build is the privacy boundary, because that one is architectural rather than operational.

SECTION 08

Scope and proof

Must build

- Versioned MQTT 5 schema with validation, retained discovery and status, and Will messages.
- 20+ emulated devices across five member processes, launched by one seeded command.
- Debt-weighted water-filling allocator, with an **independent** cap validator in a separate module that can veto any plan.
- The lease mechanism: local expiry, coordinator restart delay, and the visible countdown.
- Command, acknowledgement, heartbeat, stale detection and replan loop.
- Append-only event log with monotonic sequence numbers, and one-click replay.
- Live console with the four judge controls and the load path inspector.
- Automated tests plus **property-based tests** over the safety invariants.

Nice to have, in this order

- 1 CP-SAT deadline planner behind the existing interface. [S20]
- 2 One ESP32 with an LED on the identical contract.
- 3 OpenADR-shaped event adapter translating a capacity event into a site cap.
- 4 Downloadable incident bundle: log, plan trace, metrics, replay.
- 5 Greedy-versus-CP-SAT comparison on the same seed.

Do not build

Each of these was considered. The reason is recorded so the absence can be defended rather than apologised for.

Not building	Because
Mains switching, breaker or inverter integration	Safety — and it would make the demo a liability rather than an asset
A real Matter controller or production OpenADR endpoint	Cannot be done honestly in 48 hours. We ship interfaces and label them
Forecasting or any ML	Adds a dependency and a failure mode to buy nothing the demo needs
LLM features	There is no question in this product that a language model answers
Billing, payments, carbon credits	Unverifiable claims attached to a system that measures watts
Accounts, mobile apps, notifications	Not on the judged path
A general automation rule builder	It is a product, not a feature, and it dilutes the mechanism

Not building	Because
City-scale simulation	Claims a scale we cannot demonstrate
Peer-to-peer consensus	Research, not a weekend. The hierarchy <i>is</i> the design

Acceptance criteria

Numbered so they can be ticked in front of a judge.

- 1 All seeded devices appear within **10 s** of scenario start.
- 2 A cap change produces a validated plan within **1 s**; if CP-SAT exceeds its box, water-filling returns immediately and the UI says which ran.
- 3 Observed controlled load is below the top chord within **5 s** of a cap drop, whenever a feasible plan exists.
- 4 Commands are idempotent: replaying any command cannot toggle a device twice.
- 5 A killed member goes stale within its timeout, its last known draw stays reserved as camber, and the plan is redone conservatively.
- 6 **Killing the coordinator drives every member to its floor within one lease TTL, with zero cap violations.**
- 7 A restarted coordinator issues no lease for one full TTL, and says so on screen.
- 8 If the sum of floors exceeds the cap, the system declares INFEASIBLE and names the unserved members.
- 9 The same seed produces the same plan and a byte-identical event trace.
- 10 Every one of 1–9 has an automated test, and 3, 6 and 8 have property-based tests over generated scenarios.

Measured outputs

Metric	Definition
Cap compliance	% of samples at or below the top chord after the response grace period
Time to safe	Cap event timestamp to first verified below-cap timestamp
Lease safety violations	Instants where outstanding leases exceeded the cap. Must be 0
Acknowledgement success	Applied acks ÷ issued commands
Deadline satisfaction	Deadline jobs completed by their requested time
Recovery time	Failed-member detection to replacement plan published
Fairness	Leximin gap, plus Jain's index over delivered ÷ requested flexible energy — reported only where every denominator is valid, and described as an engineering diagnostic, never as a moral guarantee
Coordinator-loss margin	Peak observed load during a coordinator kill, as a fraction of the cap

SECTION 09

The console

One screen. It answers seven questions, and any panel that answers none of them is deleted.

- 1 What is the limit? — the **top chord**, drawn as a hard line
- 2 What are we actually drawing? — observed load, live, against that line
- 3 What can't we see? — the **camber** band, hatched, under the chord
- 4 Who has what? — five member panels with budget, draw and lease countdown
- 5 What changed, and why? — plan timeline with reason codes
- 6 Did the devices obey? — pending / verified / rejected / stale markers
- 7 Who has given up the most? — the **debt ledger**

```
+-----+
| TRUSS   site: hostel-block-c          * LEASED   rule: water-filling |
+-----+
| TOP CHORD ===== 1800 W |
| ///// camber 240 W (member D stale) ////////////////////////////////// |
| ##### observed 1512 W |
| BOTTOM CHORD ----- 820 W |
+-----+
| +-----+ +-----+ +-----+ +-----+ +-----+ |
| | A | | B | | C | | D | | E | | |
| | 312 W | | 298 W | | 340 W | | STALE | | 322 W | |
| | 4.8 s | | 4.8 s | | 4.8 s | | -- | | 4.8 s | | lease TTL |
| | debt 0 | | debt 0 | | 240 Wh | | 180 Wh | | debt 0 | |
| +-----+ +-----+ +-----+ +-----+ +-----+ |
+-----+
| PLAN TIMELINE > 19:41:02 cap_drop 5000->1800 plan-0042 |
| > 19:41:02 6 devices deferred . 0 rejected |
| > 19:41:07 verified below chord (4.9 s) |
+-----+
| [ TOP CHORD ---o--- ] [ PROTECT ] [ CHAOS ] [ REPLAY ] |
+-----+
```

The lease countdown is the centrepiece

Every member panel shows its lease TTL ticking down and resetting on renewal. It looks like a heartbeat, and that is exactly what it is. It costs almost nothing to build and it does something no other element can: it makes the safety mechanism **visible while it is working**. When the judge kills the coordinator, they do not have to take our word for what happens next. They watch five countdowns run to zero and five panels drop to their floor.

The load path inspector

Click any deferred device. A panel opens with the causal chain, read directly from the basis block that travelled with the lease — so the explanation cannot drift from the decision, because it *is* the decision.

```
washer-1 . deferred . 19:41:02
```

```

top chord          1800 W
- camber           240 W   member D stale, last draw 240 W
= allocatable      1560 W

rule               debt_weighted_water_filling
member C weight    1.24    (debt 240 Wh)
member C budget    340 W
member C floor     180 W

local decision     washer-1 (520 W) deferred
                   fridge, router, light held

deadline           09:00 . 2 of 3 slots remain . still satisfiable
debt accrued       +180 Wh

[ show event trace ] [ replay from here ]

```

Judges ask “why did it do that?”. This answers in five seconds, with arithmetic rather than adjectives, at any depth they care to push.

Colour, and one rule about it

State	Colour	Meaning
Verified	green	Commanded, acknowledged, observed
Pending	amber	Commanded, not yet acknowledged — working, not broken
Stale	hatched grey	Unknown. Reserved as camber
Rejected	blue	Device legitimately refused. Not an error
Over chord	red	Reserved exclusively for a genuine cap violation

Amber never means error. Red appears only for a real violation of I1. If red is on screen at the end of a demo run, we have a bug — not a colour choice. That is precisely why the rule exists.

Anti-requirements

No login. No settings page. No map. No “AI insights” panel. No gauge that duplicates a number already on screen. No animation that outlives its transition.

SECTION 10

Execution

Five people

	Owns	Testable output
M1	Schemas, coordinator state machine, allocator interface, water-filling, CP-SAT, integration, technical defence	<code>allocate()</code> passes property tests; coordinator completes lease/ack/replan; the architecture doc matches reality
M2	Device emulator, 20–30 seeded profiles, state machines, idempotence and failure flags	One command launches five members; every node idempotent; failure modes selectable
M3	Broker, member agent, lease expiry , event log, replay	Broker restart checklist; member self-limits with no coordinator; replay is byte-identical
M4	Console, load-path inspector, judge controls	Seven questions answered on one screen; lease countdown live; inspector reads from <code>basis</code>
M5	Chaos tooling, scenario QA, ESP32, demo assets, pitch	Six chaos commands work; three cold-start passes recorded; backup video exists

Dependency discipline

The failure mode that kills this project is M1 becoming a bottleneck while four people wait. Three rules prevent it.

- 1 The contract is frozen at hour 1** by M1 and M2 together, with one valid and one invalid example per message type committed. After that, schema changes need both of them plus a stated reason in the commit.
- 2 Everyone codes against fixtures first.** M4 builds the whole console against a static JSON snapshot and connects to live state at hour 12. M5 writes chaos commands against a stub. Nobody waits for anybody.
- 3 The allocator is a pure function**, so M1 develops and tests it with zero running infrastructure, and M4 renders its output before it is wired up.

M1 reviews rather than rewrites. A teammate's working module that M1 would have written differently ships as written. The commit history has to show five people building a system — because it will be read, and because it is true.

Forty-eight hours

Window	Objective	Deliverable and test
0–2 h	Rules confirmed, clean repo, issue board. M1+M2 freeze the six schemas. M3 launches broker. M4 console shell. M5 scenario and chaos folders.	Two processes exchange a validated heartbeat and command. README states the safety boundary on day one.
2–6 h	The vertical slice. Five devices, member agent with local floor, twin plus water-filling, console on fixtures.	A CLI cap change moves two devices, both acknowledge, cap check passes. Recorded.
6–12 h	Leases. TTL, local expiry, coordinator-restart delay. Expand to 20–30 nodes with failure flags. Versioning and replan.	Killing the coordinator drives every member to floor with zero cap violations. The test that matters most.
12–18 h	Camber, debt weighting, evidence. Independent validator. Idempotence tests. Replay. Load-path inspector. Chaos commands.	Allocator and validator disagree on nothing across the fixture set. Replay is byte-identical.
18–24 h	Integration and Review 1 hardening. Three scripted scenarios. One-screen hierarchy fixed.	Review 1: live messages, complete loop, lease countdown visible, exact roadmap. Tag a checkpoint.
24–30 h	Property tests and fault injection. Hypothesis over the invariants. Every chaos command run. Hardware go/kill decision.	The system never reports false compliance under any injected fault.
30–36 h	CP-SAT only if everything above is green. Otherwise UX and evidence. Freeze controls and layout.	Review 2 build runs from a clean start script. Three consecutive cold-start passes.
36–42 h	Feature freeze. Backup video and screenshots. All five rehearse questions and subsystem explanations.	Versioned release, offline run instructions, evidence folder, demo under 2:50.
42–48 h	Bug fixes with rollback only. Rest in rotation.	Final build hash, local backup, alternate laptop copy, live <i>and</i> recorded demo paths.

Kill points

Decisions with a time and an owner, made in advance so they are not made at three in the morning.

Hour	Question	If no
6	Does the vertical slice work end to end?	Cut to three members and two device types. Nothing else matters until this is true
12	Do leases expire correctly with the coordinator dead?	Stop all feature work. This is the project
24	Are the chaos commands real?	Cut CP-SAT and the ESP32 outright
30	Is CP-SAT beating water-filling on the fixture set?	Cut it. Say so in the pitch as a decision, not an omission

Hour	Question	If no
36	Three consecutive cold-start passes?	Freeze immediately, no exceptions

SECTION 11

The demo

Three minutes, six beats. Beats 5 and 6 are the ones no other team will have.

0:00–0:20 • The moment

“A hostel block loses grid power at 19:40. The inverter can hold 1.8 kW. The block is drawing 4.6. Today, three things can happen: the inverter trips and everyone loses everything, the warden cuts a whole floor, or somebody spends twenty minutes walking corridors unplugging things.”

0:20–0:40 • Why monitoring is not enough

Switch to **Uncoordinated** and drag the chord to 1.8 kW. The chord turns red; an overload timer runs. “Monitoring tells us we are failing. It does not decide, act, or prove recovery.”

0:40–1:00 • The insight

“Most of that load is not urgent, and the building already knows it. But none of it is expressible to whatever decides who loses power — and nobody should have to publish a device list to keep their fridge running. So each home publishes five numbers instead. Nothing else leaves the home.”

1:00–2:10 • The live loop

- 1 **Judge drags the chord to 1.8 kW.** Plan in under a second. Water heaters and the washer defer; fridge, router and the protected device hold. Amber pending dots turn to green verified ticks. Observed load crosses below the chord. **Time to safe: 4.9 s.**
- 2 **Judge clicks “protect this washer”.** The plan changes *elsewhere*. The cap is never violated. “It bent; it did not break the constraint.”
- 3 **Judge kills member C.** Its panel goes hatched. The camber band visibly grows — “its last known draw is still reserved, because unknown is not zero” — and another member’s flexible slot moves.
- 4 **Click the deferred washer.** The load path opens. “That is not a log message. That is the arithmetic that made the decision, travelling with the decision.”

2:10–2:35 • The two beats nobody else has

5 — “Kill the coordinator.”

Five lease countdowns run to zero. Five members step down to their floors. No overload. No flapping. The site gets *quieter*.

“A budget here is not a command — it is a lease with a six-second life. If it is not renewed, every home falls back to its safe floor on its own, without being told. Safety is a property of the protocol, not of the planner. Kill the brain and this gets safer, not more dangerous.”

Restart it. It refuses to issue a lease for one full TTL and says so on screen. “It cannot know what it promised before it died, so it waits.”

6 — “Now set the chord below the sum of the safety floors.”

The system goes INFEASIBLE and names the members it cannot serve.

“It will not report a compliance it did not achieve. A system that lies about its constraint is worse than no system, because someone would trust it.”

2:35–3:00 · Boundary and next step

“Truss never switches mains voltage. It is not a smart panel, not certified, and it makes no savings claims. It is the coordination layer between a grid event and a home's own gateway — the layer that turns the smart meters India is already installing into something that can act fairly and prove it did. Next: one real Home Assistant household on the same contract, and policy validation with a hostel operator.”

The wow moment, named

It is not the slider. Every energy project has a slider. It is killing the coordinator and watching the system get safer — a distributed-systems result, visible without explanation, and physically unavailable to a team that only built a dashboard.

Failure proofing

If	Then
Internet is down	Nothing changes. Broker, allocator, console and devices are all on the LAN
A laptop cannot join	Launch that member's IDs on the primary laptop. Single-laptop profile is rehearsed
ESP32 is absent	Software node on the identical contract, on-screen LED
ESP32 dies mid-demo	Use it as beat 3. A planned failure demo absorbs an unplanned one
Broker restarts	Leases expire, members floor, retained state restores. Narrate it
The live run fails entirely	Replay. One click, deterministic, animates the exact stored event log
Everything fails	90-second recording, labelled as a recording, plus screenshots of all six beats

SECTION 12

Pitch and defence

10-second hook

A truss carries a load no single beam could, by giving every member only what it can bear. Truss does that for a shared electrical feeder — and if you kill its coordinator, it gets safer, not more dangerous.

The three sentences to land

- 1 **“The flexibility is already there. What’s missing is a way to express it without surrendering privacy.”**
- 2 **“A budget isn’t a command, it’s a lease — so safety is a property of the protocol, not the planner.”**
- 3 **“A system that can’t meet its constraint has to say so, not fake it.”**

Ten questions we will get

Question	Answer
Isn’t this just load shedding?	Load shedding cuts by geography. We allocate by declared need, guarantee a floor, and remember who gave up what so the same home isn’t the loser twice. Rotational shedding cannot tell a CPAP machine from a water heater.
Why not just sort by priority?	Then the lowest-priority home is the designated loser every round, forever. Water-filling gives max-min fairness and debt weighting repays sacrifice. Fairness here is an algorithm with a theorem, not an adjective.
Why water-filling over an optimiser?	Deterministic, $O(n \log n)$, provably max-min fair, explainable in one sentence. CP-SAT sits behind the same interface and must beat it on the fixture set or we ship without it.
What if the optimiser is wrong or times out?	An independent validator re-checks every plan against the cap and can veto it. A vetoed or timed-out plan is discarded and water-filling’s plan is issued. The console names which rule ran.
What if the network or coordinator dies?	Leases expire and every member falls to its floor locally. No cloud on the judged path. A restarted coordinator waits one full TTL, because it can’t know what it promised before it died.
How do you know a device obeyed?	We don’t assume. Every command is versioned and expiring; every device acknowledges with its <i>observed</i> state; the coordinator compares desired against observed watts. Missing acks reserve conservative power and trigger a replan.
Isn’t a missing device just zero load?	No, and that assumption is the most dangerous thing this system could make. A silent member keeps its last known draw reserved as camber, and you can watch the reserve band grow on screen.

Question	Answer
How is this fair, really?	Three commitments: a floor nobody crosses, equal treatment of equal need by construction, and debt that carries between rounds. Ostrom's design principles rendered as an allocation function. The fairness index is an engineering diagnostic, not a moral guarantee.
How does it scale?	The coordinator sees offers, not devices, so its problem is $O(\text{members})$. That's the same boundary that gives privacy — the method-of-sections cut. Nesting coordinators is next and we haven't built it.
What would production need that you don't have?	Per-device identity, TLS, broker ACLs, key rotation, signed firmware, audit retention, certified actuation and real standards adapters. We built none of those in 48 hours and we're not going to imply otherwise.

Strongest closing

We can show you a system that works. More usefully, we can show you what happens when it doesn't: kill the coordinator and every home falls back to its safe floor on its own. That is the difference between a demo and a system.

What we will not say

No market-size slide. No “AI-powered”. No savings percentage. No “revolutionise”. No claim of a Matter or OpenADR implementation. No user count, no pilot, and no partner we do not have.

SECTION 13

Risks

#	Risk	L	I	Mitigation
R1	M1 becomes the bottleneck and four people wait	High	Fatal	Contract frozen at hour 1; everyone builds against fixtures; allocator is a pure function
R2	Distributed integration eats the event — five subsystems that never connect	High	Fatal	Vertical slice by hour 6 is a hard kill point. Three members is an acceptable retreat; six disconnected demos is not
R3	Lease semantics subtly wrong — expiry fires late, or a member over-draws	Med	Fatal	Hour-12 kill point is exactly this test. Property tests on I3. Monotonic clocks only
R4	CP-SAT consumes hours and delivers nothing	Med	High	Behind an interface and time-boxed. Hour-30 kill point. Cutting it is a defensible answer
R5	Demo laptop cannot join the LAN at the venue	Med	High	Single-laptop profile rehearsed as often as the distributed one. Own router <i>and</i> hotspot
R6	ESP32 fails at the venue	Med	Low	Bonus only. If it dies mid-demo it becomes beat 3
R7	Console becomes a second project	Med	Med	Seven-questions rule; anything answering none is deleted. Frozen at hour 36
R8	Judges read it as “just load shedding”	Med	High	Beats 5 and 6 exist for this. Lead with the lease and the refusal, not the slider
R9	Someone claims Matter/OpenADR compliance under pressure	Low	Fatal	The boundary is on slide one and in the README. Rehearsed as a scripted sentence
R10	Scope creep from mentor feedback at Review 1	High	Med	Feedback is logged, not implemented. Only core-loop fixes are actioned before Review 2
R11	Event dates or submission deadline missed	Low	Fatal	Open items confirmed directly, today, with a written record of who said what
R12	Fatigue-driven breakage in the last six hours	High	High	Hours 42–48 are bug fixes with rollback only. Rest in rotation. Schema and allocator frozen

The three that actually decide the outcome

- **R2** — if the vertical slice is not alive at hour 6, nothing else in this document matters. Everything is subordinate to that.
- **R3** — the lease mechanism is the difference between this project and a nicer dashboard. Without it we have an ordinary submission with good vocabulary.
- **R8** — the idea is genuinely distinguishable, but only if we lead with the distinguishing parts. The slider is the setup; the coordinator kill and the refusal are the pitch.

Competition constraints

Code2Create requires projects to be built during the event, from scratch, with no pre-built projects, prior commits or reused code. AI tool use is permitted; hardware is allowed but self-supplied. [S17][S18]

What that does not forbid: deciding what to build, freezing interfaces on paper, learning the tools with disposable exercises, brand and pitch material, and rehearsal. The preparation repository holds plans, brand, protocol specification and research — and no implementation code. The competition repository is created after the clock starts.

Four things to confirm directly with organisers — do not infer any of them

The event dates. The official gravITas page and ACM-VIT's promotional material disagreed (6–8 vs 7–9 September). [S17][S19]

Idea submission state and deadline. Sources gave conflicting internal and external deadlines.

What “from scratch” permits. Ask explicitly whether written interface specifications prepared in advance are acceptable. Until answered, assume the strictest reading.

Judging weights. The official page listed criteria as TBD.

SECTION 14

Sources

Confidence vocabulary

- **Verified** — the linked source directly supports the limited claim made.
- **Strong inference** — two credible facts align, but no single source joins them. Labelled wherever used.
- **Speculation** — labelled, and excluded from every strategy claim.

Three standing rules. Commercial product pages establish that a **feature exists**, not independent performance. Standards references establish **protocol direction**, not interoperability or certification of this prototype. Problem-class evidence establishes that **the problem is real**, not demand or savings for Truss.

Core mechanism

ID	Source	What it supports
S01	Gray & Cheriton, <i>Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency</i> , SOSP '89	The lease mechanism, fail-safe expiry, and the coordinator-restart rule. Non-Byzantine failures affect performance, not correctness; short terms bound the damage
S02	Max-min fairness / leximin / water-filling literature (Bertsekas & Gallager; Radunović & Le Boudec)	Water-filling yields the max-min fair allocation; max-min fair allocations are the lexicographically maximal feasible ones
S07	Packetized Energy Management — Almassalkhi, Frolik, Hines et al.	Nearest prior art: anonymous asynchronous packet requests, statistical fairness, preserved privacy and quality of service
S08	Ostrom, <i>Governing the Commons</i> (1990); Ostrom Workshop design principles	Boundaries, congruence, collective choice, monitoring, graduated sanctions, conflict resolution — the frame for the member boundary and the debt ledger

Standards

ID	Source	What it supports
S03	OASIS MQTT 5.0 specification	Lightweight pub/sub for IoT. Sessions, QoS, message expiry, Will messages
S04	Eclipse Mosquitto documentation	Open-source lightweight MQTT 5/3.x broker
S05	OpenADR Alliance — OpenADR 3.x, VTN/VEN architecture	Standardised demand-response event exchange; 3.0 moves to REST with webhooks; 3.1 is not backwards compatible. Upstream signal direction only
S06	Connectivity Standards Alliance — Matter 1.5	Electrical energy tariff device type; Commodity Tariff, Commodity Price, Electrical Grid Conditions, Energy Preference, Power Topology clusters. Not evidence that we implement Matter

Problem class and prior art

ID	Source	What it supports
S09	IEA, <i>Electricity 2026</i> — flexibility analysis	Flexible demand systematically under-used
S10	IEA, <i>Efficient Grid-Interactive Buildings in India</i>	Grid-interactive buildings as an Indian opportunity joining efficiency, digital technology and peak shifting
S11	IEA, <i>Scaling Up Demand Flexibility</i>	Reliability, cost, renewable integration, network-constraint benefits
S12	India RDSS programme reporting	25 crore smart meters, March 2028 deadline, ~₹2.83 lakh crore sanctioned. Supports “measurement arriving, coordination missing”. Does not establish demand for Truss
S13	SPAN official material	Circuit control, load priority, dynamic EV charging. Feature existence only
S14	Lumin official material	Power limit, load management, scheduling, local control. Feature existence only
S15	Home Assistant energy documentation	Multi-brand energy visibility and device integration
S16	Home Assistant MQTT documentation	Local broker, discovery, MQTT 5 integration

Tooling and event

ID	Source	What it supports
S20	Google OR-Tools documentation	CP-SAT suitability for discrete on/off loads, deadlines, minimum runs, capacity
S21	Hypothesis documentation	Property-based test generation over the safety invariants
S22	Wokwi documentation	ESP32 Wi-Fi/MQTT simulation. Rehearsal aid only — its public gateway cannot reach our LAN
S17	Official graVITas '26 / Code2Create 7.0 page	Venue, duration, team size, hardware self-supply, from-scratch rule, AI allowance. Criteria listed as TBD. Date conflict unresolved
S18	Unstop organiser listing	Registration, eligibility, round sequence, eliminatory prototype review, prohibition on pre-built projects and prior commits
S19	ACM-VIT public material	Open Innovation framing, submission deadlines, and a conflicting 7–9 September date. Confirm directly

What we must never say

That we implement OpenADR or Matter. We ship one OpenADR-*shaped* fixture and a Matter adapter *interface*.

That Truss is cheaper, greener or more efficient than any named product. We have measured none of that.

That Packetized Energy Management is our idea. It is prior art, and naming your nearest neighbour in the literature is a strength.

That any Indian policy document mandates or endorses this. Draft consultation material is not law.